Estado del Arte: Aumentación de Datos con TAOS, EDA y SMOTE-Tomek, Clasificación con BERT y Evaluación con LLM para Requerimientos No Funcionales

Andres Lino Montes Mamani, Cristian Cabana Sulca, Alessandro Pastor Mamani Mamani
Escuela Profesional de Ingeniería de Sistemas
Universidad Peruana Unión, Juliaca, Perú
Asesor: Mg. Esteban Tocto Cano

***Resumen*—La clasificación automática de requerimientos no funcionales (NFR) sigue limitada por el desbalance extremo de clases: en la distribución original que reporta el trabajo TAOS, la clase mayoritaria tiene 557 muestras y la menor solo 2 (278.5:1). Este artículo revisa el estado del arte necesario para comparar tres métodos de aumentación de datos (TAOS, EDA y SMOTE-Tomek) entrenando BERT con cada uno, contrastar esos modelos con LLM usados como clasificadores, y ampliar después el conjunto de datos con requerimientos obtenidos de GitHub para repetir el flujo. Se comparan doce estudios y se identifican cuatro brechas: TAOS solo se contrasta con un BERT sin aumentación y su propia tabla muestra que el balanceo es parcial; los estudios usan conjuntos y clases heterogéneos; no se identificó una comparación entre LLM y BERT entrenado con distintas aumentaciones sobre las 12 clases; y falta un flujo iterativo de ampliación del conjunto con datos de GitHub. Se propone un flujo experimental que cierra estas brechas.**

***Index Terms*—requerimientos no funcionales, desbalance de clases, aumentación de datos, BERT, modelos de lenguaje de gran escala, minería de GitHub**

I. Introducción

Los requerimientos no funcionales (NFR) describen atributos de calidad como seguridad, desempeño, mantenibilidad o portabilidad, y su identificación en los documentos de especificación (SRS) es una tarea central de la ingeniería de requerimientos. Desde el trabajo pionero de Cleland-Huang et al. [1], que dio origen al conjunto PROMISE NFR [2], la clasificación automática de NFR se ha abordado con aprendizaje automático clásico [3], [4], aprendizaje profundo [5] y transformers [6], [7].

El obstáculo persistente es el desbalance de clases. En la distribución original que reportan Ben Ammar y Almatrafi [8], la clase funcional (F) tiene 557 muestras y portabilidad (PO) solo 2, una razón de 278.5:1. Con tan pocos ejemplos, un clasificador basado en BERT tiende a favorecer las clases frecuentes y a no detectar las minoritarias.

Este estado del arte respalda un trabajo con cinco pasos: (1) aumentar el conjunto de datos con TAOS, EDA y SMOTE-Tomek; (2) entrenar BERT con cada versión; (3) medir esos modelos frente a uno o varios LLM usados como clasificadores; (4) construir un aplicativo con los modelos ganadores; y (5) ampliar el conjunto con requerimientos de GitHub y repetir el flujo. La Fig. 1 resume los bloques de literatura que se revisan: la Sección III presenta los fundamentos, la IV los métodos de aumentación, la V los LLM como clasificadores, la VI la ampliación con datos de GitHub, la VII compara los estudios, la VIII expone las brechas y la IX la propuesta.

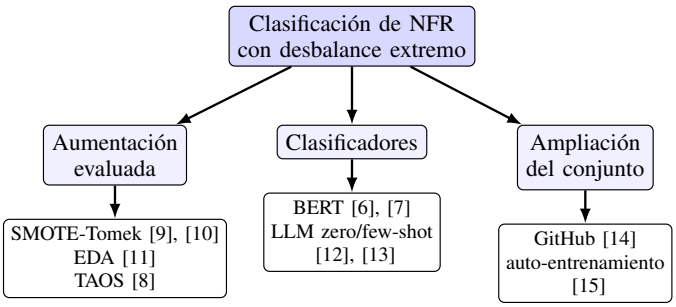


Fig. 1. Bloques de literatura revisados en este estado del arte.

II. Alcance y metodología

Este estado del arte se apoya en el protocolo de la revisión sistemática de la tesis (PRISMA 2020 [16]); el proceso de selección se resume en la Fig. 2 y los vínculos entre estudios en el mapa de la Fig. 5. Se priorizan artículos de revista y de conferencia y los preprints se señalan como tales. Cada fuente citada se verificó contra su registro (DOI/Crossref, ACL Anthology o arXiv). De TAOS [8] se revisó el texto completo; de los demás estudios se leyeron los resúmenes, por lo que los resultados de la Tabla I son los que declaran sus autores en ellos.

Esta revisión *no es exhaustiva*: las afirmaciones "no se identificó" se refieren a las fuentes revisadas y no a toda la literatura. Las búsquedas en Scopus, Web of Science, IEEE Xplore y ACM DL corresponden a la RSL y deben completarse con sus conteos antes de sostener una ausencia de estudios.

Imagen pendiente
`recursos/prisma_nfr.png`

Fig. 2. Diagrama de flujo PRISMA de la selección de estudios (ver la RSL).

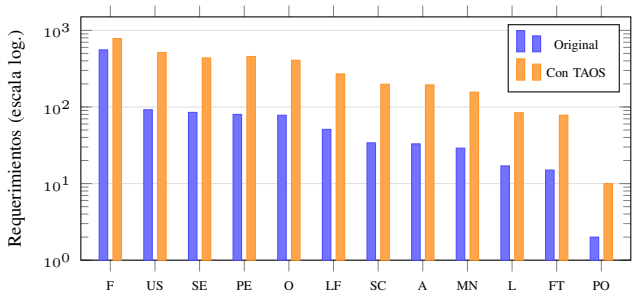


Fig. 3. Distribución de clases antes y después de TAOS (Tabla III de [8]).

III. Fundamentos

III-A. Clasificación automática de NFR

Los primeros trabajos usaron clasificadores supervisados sobre representaciones léxicas [1], [3]; después aparecieron enfoques interpretables con análisis de dependencias [4] y ensambles de redes profundas. Rahimi et al. [5] combinaron LSTM, BiLSTM, GRU y CNN en sistemas de una y dos fases sobre PROMISE y reportaron 95.7 % de exactitud en la clasificación binaria y 93.4 % en la multi-clase de 17 clases. NoRBERT [6] introdujo la transferencia con BERT [7] para esta tarea, y Binkhonain y Zhao [17] revisan los algoritmos usados para identificar y clasificar NFR.

III-B. Desbalance de clases

El desbalance degrada el desempeño sobre las clases minoritarias y se estudia extensamente en aprendizaje profundo [18]. Una estrategia es generar datos sintéticos de las clases minoritarias, que es el eje de este trabajo.

III-C. Conjunto de datos de referencia

La Fig. 3 muestra la distribución de clases que reportan Ben Ammar y Almatrafi en su Tabla III, antes y después de aumentar con TAOS. Sus conteos originales suman 1,073, la partición de entrenamiento de los 1,534 requerimientos del conjunto (461 para prueba) [8].

IV. Métodos de aumentación de datos

IV-A. SMOTE-Tomek

SMOTE sintetiza ejemplos minoritarios interpolando vecinos en el espacio de características [19], y su combinación con enlaces de Tomek limpia la frontera entre clases [9]. Al operar sobre vectores, no produce requerimientos legibles. En requerimientos, Or [10] aplicó SMOTE-Tomek con validación cruzada estratificada al conjunto PROMISE de 969 requerimientos en la tarea binaria funcional/no funcional: la regresión logística alcanzó 76.16 % de exactitud frente a 59.85 % de la línea base. Rubaidi et al. [20] tratan el desbalance en la misma tarea binaria con algoritmos de aprendizaje automático. En ambos casos no se evaluó el problema de 12 clases.

IV-B. EDA

EDA [11] modifica el texto con cuatro operaciones simples (reemplazo por sinónimos, inserción, intercambio y eliminación aleatorios) y es una referencia de la aumentación de texto en NLP; existen revisiones amplias de estas técnicas [21], [22]. En NFR, Limaylla-Lunarejo et al. [23] usaron sobremuestreo y submuestreo sobre PROMISE_exp en español, y las clases subrepresentadas (portabilidad y tolerancia a fallos) se beneficiaron aunque el desempeño global bajó. En las fuentes revisadas no se identificó una evaluación de EDA para las 12 clases con BERT.

IV-C. TAOS y TextAttack

TextAttack [24] es un marco que, además de ataques adversarios, ofrece recetas de aumentación de datos. Sobre él, Ben Ammar y Almatrafi propusieron TAOS [8]: con BERT base uncased, 1,073 requerimientos de entrenamiento y 461 de prueba, elevan el F1 ponderado de 0.75 a 0.93 y el F1 macro de 0.44 a 0.89. El modelo base no reconocía las clases MN, L, FT y PO, y con TAOS obtuvieron F1 entre 0.50 y 1.00. Dos observaciones limitan la lectura de estos resultados. Primero, TAOS se compara únicamente con un BERT sin aumentación; EDA figura solo en su bibliografía. Segundo, aunque el texto indica que la aumentación continúa hasta que las clases minoritarias alcanzan un tamaño comparable al de la mayoritaria, su Tabla III muestra que PO pasa de 2 a solo 10 muestras (5.0×) mientras F sube a 782, es decir, una razón de unos 78:1: el balanceo es parcial (Fig. 3). Los autores publican el código en GitHub.

V. LLM como clasificadores de requerimientos

Varios trabajos evalúan LLM directamente como clasificadores, sin entrenamiento específico. El-Hajjami et al. [12] evaluaron gpt-3.5-turbo y gpt-4 en zero-shot y few-shot sobre cinco conjuntos y los compararon con SVM y LSTM: no hay una técnica mejor para todas las clases, y el few-shot ayuda sobre todo cuando el zero-shot es bajo. Alhoshan et al. [13] probaron Bloom, Gemma y Llama en más de 400 experimentos sobre PROMISE NFR, Functional-Quality y SecReq, y concluyen que el diseño del prompt y la arquitectura importan siempre, mientras que el efecto del conjunto de datos depende de la complejidad de la tarea. Tang et al. [25] muestran que exceder los ejemplos en el prompt puede degradar el desempeño y que seleccionar ejemplos por TF-IDF mejora el resultado, superando en 1 % al estado del arte en la clasificación funcional/no funcional. Kassab y Belbach [26] combinan cuatro LLM con votación ponderada y apilamiento

sobre 984 requerimientos, y Sakib et al. [27] comparan RoBERTa, CodeBERT, Gemma-2, Phi-3, Mistral y Llama-3.1, con Gemma-2 en el primer lugar (F1 de 0.88). Estos estudios comparan LLM con métodos clásicos u otros LLM, pero no con BERT entrenado con distintas aumentaciones.

Los LLM también se han usado como *generadores* de datos: AugGPT [28] y Møller et al. [29] en clasificación de texto general, y ChatNRC [30] y Aldhafer et al. [31] en requerimientos. Esa línea queda fuera del alcance de este trabajo, donde los LLM solo clasifican.

VI. Ampliación del conjunto con datos de GitHub

GitReq [14] (preprint de 2026) reúne 6,302 requerimientos validados por expertos, minados de 55,588 candidatos en 4,080 repositorios de GitHub, con ocho categorías alineadas con ISO/IEC 25010 (desempeño, seguridad, portabilidad, disponibilidad, tolerancia a fallos, escalabilidad, mantenibilidad y una base funcional); evalúan cuatro LLM en zero-shot y el mejor (GPT-5.2) alcanza un F1 macro de 0.641. Otros trabajos extienden conjuntos de NFR con fuentes abiertas: Sakib et al. [27] recolectaron NFR de cartas de proyecto y de documentación de software libre, y Pérez-Verdejo et al. [32] proponen un enfoque automatizado para clasificar requerimientos de calidad a partir de issues de GitHub.

Para usar el propio clasificador en la ampliación, el antecedente es el aprendizaje semi-supervisado: Casamayor et al. [15] lo aplicaron a la identificación de NFR aprovechando requerimientos sin etiquetar. Debe considerarse que las etiquetas asignadas por un modelo pueden arrastrar sus propios errores al reentrenar, y que la taxonomía de GitReq (8 categorías) no coincide con las 12 clases del conjunto de referencia, por lo que se necesita un mapeo y validación humana.

VII. Análisis comparativo

La Tabla I sintetiza los estudios. Los resultados no son comparables entre sí: cambian el conjunto (969, 984 o 1,534 requerimientos, PROMISE_exp traducido, GitReq), el número de clases (2, 8, 10, 12 o 17), el clasificador y la métrica (exactitud, F1 ponderado o macro). Y los estudios sobre aumentación y sobre LLM son líneas separadas: ninguno cruza ambas.

VIII. Brechas identificadas

Cada brecha se formula respecto a las fuentes revisadas (Tabla II).

IX. Propuesta de flujo experimental

La Fig. 4 presenta el flujo que cierra las brechas anteriores. Se mantiene fijo el conjunto de referencia y su partición (1,073 para entrenamiento y 461 para prueba) y se aumenta solo el conjunto de entrenamiento, con el fin de evitar fuga de información hacia la prueba. Además de TAOS, EDA y SMOTE-Tomek se incluye un control sin aumentación, indispensable para medir el aporte de cada método. Se entrena BERT base uncased con cada versión y se evalúa en la misma partición de prueba con F1 macro, ponderado y por clase, poniendo atención en MN, L, FT y PO. En paralelo, uno o varios LLM clasifican esa misma partición en zero-shot y few-shot. Los modelos ganadores alimentan un aplicativo de predicción y, a continuación, se minan requerimientos de GitHub, se etiquetan con el modelo, se validan por muestreo, se agregan al conjunto y se repite el flujo.

X. Conclusiones

La literatura ofrece por separado métodos de aumentación, clasificadores basados en BERT, evaluaciones de LLM como clasificadores y conjuntos minados de GitHub, pero no los combina. TAOS [8] es el avance más reciente sobre el problema de 12 clases, aunque se contrasta solo con un BERT sin aumentación y su balanceo es parcial. La propuesta compara TAOS, EDA y SMOTE-Tomek bajo un mismo protocolo, contrasta los modelos con LLM y establece un flujo iterativo de ampliación con datos de GitHub.

Limitación. Los resultados de la Tabla I provienen de resúmenes y no se han replicado; varios estudios son preprints; y la búsqueda sistemática y su registro PRISMA deben completarse antes de defender las brechas.

Referencias

[1] J. Cleland-Huang, R. Settimi, X. Zou, and P. Solc, "Automated classification of non-functional requirements," *Requirements Engineering*, vol. 12, no. 2, pp. 103–120, 2007.

[2] J. Cleland-Huang, S. Mazrouee, H. Liguo, and D. Port, "nfr," Zenodo dataset, 2007.

[3] Z. Kurtanović and W. Maalej, "Automatically classifying functional and non-functional requirements using supervised machine learning," in *2017 IEEE 25th International Requirements Engineering Conference (RE)*, 2017, pp. 490–495.

[4] F. Dalpiaz, D. Dell'Anna, F. B. Aydemir *et al.*, "Requirements classification with interpretable machine learning and dependency parsing," in *2019 IEEE 27th International Requirements Engineering Conference (RE)*, 2019, pp. 142–152.

[5] N. Rahimi, F. Eassa, and L. Elrefaei, "One- and two-phase software requirement classification using ensemble deep learning," *Entropy*, vol. 23, no. 10, p. 1264, 2021.

[6] T. Hey, J. Keim, A. Koziolek *et al.*, "NoRBERT: Transfer learning for requirements classification," in *2020 IEEE 28th International Requirements Engineering Conference (RE)*, 2020, pp. 169–179.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171–4186.

[8] B. Ben Ammar and N. F. Almatrafi, "Classification of requirements from software requirement specification documents: TAOS: A novel TextAttack oversampling approach for fine-grained classification of software requirements," *Engineering, Technology & Applied Science Research*, vol. 15, no. 6, pp. 29 872–29 879, 2025.

[9] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD Explorations Newsletter*, vol. 6, no. 1, pp. 20–29, 2004.

[10] B. Or, "Improving requirements classification with SMOTE-Tomek preprocessing," arXiv:2501.06491, 2025.

[11] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 6382–6388.

[12] A. El-Hajjami, N. Fafin, and C. Salinesi, "Which AI technique is better to classify requirements? an experiment with SVM, LSTM, and ChatGPT," arXiv:2311.11547, 2023.

Tabla I
ESTUDIOS REVISADOS: AUMENTACIÓN, LLM COMO CLASIFICADOR Y DATOS DE GITHUB

Estudio	Año	Enfoque	Datos y tarea	Resultado reportado	Observación
Or [10]	2025 (preprint)	SMOTE-Tomek con validación cruzada estratificada	PROMISE, 969 req.; funcional / no funcional	Exactitud 76.16 % (RL) frente a 59.85 % base	Solo binaria
Rubaidi et al. [20]	2025	ML con manejo del desbalance	Funcional / no funcional	No verificado (resumen no accesible)	Solo binaria
Ben Ammar y Almatrafi [8]	2025	TAOS: aumentación con TextAttack; BERT base uncased	1,534 req.; 12 clases	F1 ponderado $0.75 \rightarrow 0.93$; F1 macro $0.44 \rightarrow 0.89$	Solo frente a BERT sin aumentación; PO pasa de 2 a 10
Limaylla-Lunarejo et al. [23]	2025	Sobremuestreo y submuestreo; seis algoritmos	PROMISE_exp en español	F1 82 % (RL) y 81 % (NB) en el conjunto traducido	Balanceo clásico; sin BERT
Qin y Peng [30]	2025	ChatNRC: LLM generador, discriminador y clasificador	PROMISE-NFR; 4 categorías	F1 ponderado 0.96	LLM como generador; sin las clases raras
Aldhafer et al. [31]	2026	Aumentación adaptativa con BiGRU	PROMISE (10 clases) y EHR	Macro-F1 +6.7 %	Clasificador BiGRU, no BERT
El-Hajjami et al. [12]	2023 (preprint)	GPT-3.5 y GPT-4 zero/few-shot frente a SVM y LSTM	5 conjuntos de requerimientos	Sin técnica mejor para todas las clases	Sin BERT aumentado
Alhoshan et al. [13]	2025 (preprint)	Bloom, Gemma y Llama; +400 experimentos	PROMISE NFR, Functional-Quality, SecReq	El prompt y la arquitectura importan siempre	Sin aumentación de datos
Tang et al. [25]	2025 (preprint)	Selección de ejemplos few-shot; 7 LLM	2 conjuntos, funcional / no funcional	+1 % sobre el estado del arte	Tarea binaria
Kassab y Belbach [26]	2025	Ensamble de 4 LLM	984 requerimientos	Mejora frente a un solo modelo	Sin aumentación
Sakib et al. [27]	2025 (preprint)	Seis modelos; conjunto ampliado con cartas de proyecto y documentación abierta	NFR en subclases	Gemma-2: F1 0.88	Otro conjunto de datos
Kamal et al. [14]	2026 (preprint)	GitReq: minería de GitHub; 4 LLM zero-shot	6,302 req.; 8 categorías	GPT-5.2: F1 macro 0.641	Taxonomía distinta de las 12 clases

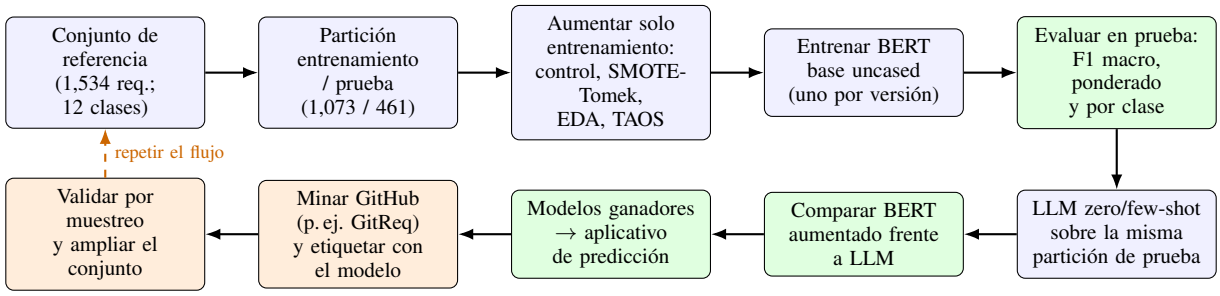


Fig. 4. Flujo experimental propuesto: aumentación, BERT, comparación con LLM, aplicativo y ampliación iterativa con GitHub.



Fig. 5. Mapa de la literatura generado a partir de un artículo semilla (p. ej. TAOS o EDA).

[13] W. Alhoshan, A. Ferrari, and L. Zhao, "How effective are generative large language models in performing requirements classification?" arXiv:2504.16768, 2025.

[14] F. Kamal, M. H. Kabir, M. R. Islam *et al.*, "GitReq: A gold standard dataset for software quality requirements," arXiv:2606.21810, 2026.

[15] A. Casamayor, D. Godoy, and M. Campo, "Identification of non-functional requirements in textual specifications: A semi-supervised learning approach," *Information and Software Technology*, vol. 52, no. 4, pp. 436–445, 2010.

[16] M. J. Page, J. E. McKenzie, P. M. Bossuyt *et al.*, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. n71, 2021.

[17] M. Binkhonain and L. Zhao, "A review of machine learning algorithms for identification and classification of non-functional requirements," *Expert Systems with Applications: X*, vol. 1, p. 100001, 2019.

[18] J. M. Johnson and T. M. Khoshgoftaar, "Survey on deep learning with class imbalance," *Journal of Big Data*, vol. 6, 2019.

[19] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

Tabla II
BRECHAS DE CONOCIMIENTO Y EVIDENCIA

ID	Brecha y evidencia
G1	**TAOS sin comparación y con balanceo parcial.** Solo se contrasta con un BERT sin aumentación, EDA se cita pero no se evalúa, y su Tabla III deja PO en 10 muestras frente a 782 de F [8].
G2	**Heterogeneidad.** Los estudios usan 969, 984 o 1,534 requerimientos y 2, 8, 10, 12 o 17 clases [5], [8], [10], [14], [26], [31].
G3	**LLM frente a BERT aumentado.** Los LLM se comparan con SVM, LSTM u otros LLM [12], [13], [27]; no se identificó una comparación con BERT entrenado con TAOS, EDA y SMOTE-Tomek sobre las 12 clases.
G4	**Ampliación iterativa con GitHub.** Existe un conjunto minado de GitHub con otra taxonomía [14] y antecedentes semi-supervisados [15], pero no se identificó un flujo que use el clasificador entrenado para ampliar el conjunto de 12 clases y repetir la evaluación.

[20] Z. S. Rubaidi, B. Ben Ammar, and M. Ben Aouicha, "Handling imbalance functional and non-functional software requirement classification based on machine learning algorithms," in *Lecture Notes in Networks and Systems*. Springer Nature Switzerland, 2025, pp. 199–209.

[21] S. Y. Feng, V. Gangal, J. Wei *et al.*, "A survey of data augmentation approaches for NLP," in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021, pp. 968–988.

[22] M. Bayer, M.-A. Kaufhold, and C. Reuter, "A survey on data augmentation for text classification," *ACM Computing Surveys*, vol. 55, no. 7, 2022.

[23] M. Limaylla-Lunarejo, N. Condori-Fernandez, M. Rodríguez Luaces, and O. Karras, "Improving the multi-class classification of non-functional requirements in Spanish: A study of dataset balancing and performance," *Empirical Software Engineering*, vol. 31, 2025.

[24] J. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 119–126.

[25] Y. Tang, D. Tuncel, C. Koerner, and T. Runkler, "The few-shot dilemma: Over-prompting large language models," arXiv:2509.13196, 2025.

[26] M. Kassab and W. Belbach, "Automating requirements classification: A systematic literature review and ensemble LLM-based approach," in *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering (EASE) Companion*, 2025, pp. 82–90.

[27] E. E. Sakib, M. A. Akib, M. M. Mazumder *et al.*, "A benchmark dataset and LLMs comparison for NFR classification with explainable AI," arXiv:2510.18096, 2025.

[28] H. Dai *et al.*, "AugGPT: Leveraging ChatGPT for text data augmentation," arXiv:2302.13007, 2023.

[29] A. G. Møller, J. A. Dalsgaard, A. Pera, and L. M. Aiello, "The parrot dilemma: Human-labeled vs. LLM-augmented data in classification tasks," arXiv:2304.13861, 2023.

[30] Y. Qin and R. Peng, "ChatNRC: A non-functional requirement classification framework based on a generative and discriminative mechanism," in *2025 32nd Asia-Pacific Software Engineering Conference (APSEC)*, 2025, pp. 467–478.

[31] O. Q. Aldhafer, H. A. Alsadiq, M. Alshaboti *et al.*, "Adaptive data augmentation techniques for software requirements classification using deep learning," *IEEE Access*, vol. 14, pp. 43 841–43 854, 2026.

[32] J. M. Pérez-Verdejo, Á. J. Sánchez-García, and J. O. Ocharán-Hernández, "Requirements and GitHub issues: An automated approach for quality requirements classification," *Programming and Computer Software*, vol. 47, no. 8, pp. 704–721, 2021.